

```
import os

print("Training:", training_dir)
print("Exists:", os.path.exists(training_dir))

print("Validation:", validation_dir)
print("Exists:", os.path.exists(validation_dir))

print("Testing:", testing_dir)
print("Exists:", os.path.exists(testing_dir))
```

```
Training: /content/chest_xray/train
Exists: True
Validation: /content/chest_xray/val
Exists: True
Testing: /content/chest_xray/test
Exists: True
```



```
import zipfile
```

```
zip_path = "/content/drive/MyDrive/archive (1).zip"  
extract_path = "/content"
```

```
with zipfile.ZipFile(zip_path, 'r') as zip_ref:  
    zip_ref.extractall(extract_path)
```

```
print("Dataset extracted successfully!")
```

... Dataset extracted successfully!

```
import os
```

```
print("Current folder:", os.getcwd())  
print("training_dir:", training_dir)  
print("Exists:", os.path.exists(training_dir))
```

```
Current folder: /content  
training_dir: /content/chest_xray/train  
Exists: True
```

```
import os
```

```
print(training_dir)
```

```
print(os.listdir(training_dir))
```

```
/content/chest_xray/train  
['PNEUMONIA', 'NORMAL']
```

```
from google.colab import drive  
drive.mount('/content/drive')
```

```
Mounted at /content/drive
```

```
show_class_sample_images(training_dir, image_count_per_category=5, save=False, dpi=100, update_image=False)
```

Number of Class: 2

Pneumonia



Normal



```
plot_params = get_reset_plot_params()

plot_params['figsize'] = (18,4)

plot_params['title_fontsize'] = 13
plot_params['label_fontsize'] = 10

plot_params['title'] = "Number of Cases"

plot_params['subplot'] = 131

show_train_val_test(training_dir, validation_dir, testing_dir, plot_params)
classes = os.listdir(training_dir)
classes = [name_correct(i) for i in classes]
```

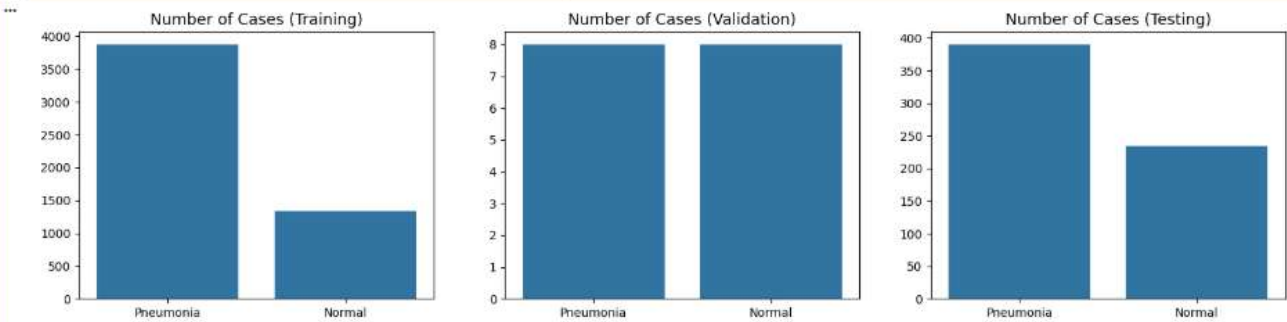


Image Preprocessing/ Augmentation/ Transformation for Training, Validation, Testing and Dataset

```
▶ # batch_size = 32

# target_size = (299, 299)
# color_mode = "rgb"

rescale = 1./255
target_size = (150, 150)
batch_size = 163
class_mode = "categorical"
# class_mode = "binary"

train_datagen = ImageDataGenerator(
    rescale=rescale,
    shear_range=0.2,
    zoom_range=0.2,
    horizontal_flip=True)

train_generator = train_datagen.flow_from_directory(
    training_dir,
    target_size=target_size,
    class_mode=class_mode,
    batch_size=batch_size,
    shuffle=True)

validation_datagen = ImageDataGenerator(rescale=rescale)

validation_generator = validation_datagen.flow_from_directory(
    validation_dir,
    target_size=target_size,
    class_mode=class_mode,
    batch_size=dir_file_count(validation_dir),
    shuffle = False)

test_datagen = ImageDataGenerator(rescale=rescale)

test_generator = test_datagen.flow_from_directory(
    testing_dir,
    target_size=target_size,
    class_mode=class_mode,
    batch_size=dir_file_count(testing_dir),
    shuffle = False)
```

```
*** Found 5216 images belonging to 2 classes.
Found 16 images belonging to 2 classes.
Found 624 images belonging to 2 classes.
```

```

print("Getting Base Model", date_time(1))
# model = get_model()
model = get_conv_model()
# model = keras.models.load_model("data/output/models/2018-12-15 00-26-45/13-val_acc-0.70-val_loss-0.58.hdf5")

```

Getting Base Model Timestamp: 2026-07-19 05:40:15

/usr/local/lib/python3.12/dist-packages/keras/src/layers/convolutional/base_conv.py:113: UserWarning: Do not pass an `input`
 super().__init__(activity_regularizer=activity_regularizer, **kwargs)

Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 150, 150, 16)	448
conv2d_1 (Conv2D)	(None, 150, 150, 16)	2,320
max_pooling2d (MaxPooling2D)	(None, 75, 75, 16)	0
conv2d_2 (Conv2D)	(None, 75, 75, 32)	4,640
conv2d_3 (Conv2D)	(None, 75, 75, 32)	9,248
max_pooling2d_1 (MaxPooling2D)	(None, 37, 37, 32)	0
conv2d_4 (Conv2D)	(None, 37, 37, 64)	18,496
conv2d_5 (Conv2D)	(None, 37, 37, 64)	36,928
max_pooling2d_2 (MaxPooling2D)	(None, 18, 18, 64)	0
conv2d_6 (Conv2D)	(None, 18, 18, 96)	55,392
conv2d_7 (Conv2D)	(None, 16, 16, 96)	83,040
max_pooling2d_3 (MaxPooling2D)	(None, 8, 8, 96)	0
conv2d_8 (Conv2D)	(None, 8, 8, 128)	118,720
conv2d_9 (Conv2D)	(None, 6, 6, 128)	147,584
max_pooling2d_4 (MaxPooling2D)	(None, 3, 3, 128)	0
flatten (Flatten)	(None, 1152)	0
dense (Dense)	(None, 64)	73,792
dropout (Dropout)	(None, 64)	0
dense_1 (Dense)	(None, 2)	130

Total params: 542,738 (2.07 MB)

Trainable params: 542,738 (2.07 MB)

Non-trainable params: 0 (0.00 B)

```
class_weight = get_weight(train_generator.classes)
print(class_weight)
```

```
{0: np.float64(1.9448173005219984), 1: np.float64(0.6730322580645162)}
```



```

print("Starting Training Model", date_time(1))

steps_per_epoch = len(train_generator)
validation_steps = len(validation_generator)

optimizer = Adam(learning_rate=0.001)
loss = 'categorical_crossentropy'
metrics = ['accuracy']
epochs = 100

# Compile model
model.compile(
    optimizer=optimizer,
    loss=loss,
    metrics=metrics
)

# Train model
history = model.fit(
    train_generator,
    steps_per_epoch=steps_per_epoch,
    epochs=epochs,
    verbose=2,
    callbacks=callbacks,
    validation_data=validation_generator,
    validation_steps=validation_steps,
    class_weight=class_weight
)

print("Completed Model Training", date_time(1))

```

... Starting Training Model Timestamp: 2026-07-19 06:19:55
Epoch 1/100

Epoch 1: val_accuracy did not improve from 0.68750
32/32 - 447s - 14s/step - accuracy: 0.8671 - loss: 0.3059 - val_accuracy: 0.6250 - val_loss: 0.6337 - learning_rate: 0.0010
Epoch 2/100

Epoch 2: val_accuracy did not improve from 0.68750
32/32 - 449s - 14s/step - accuracy: 0.9089 - loss: 0.2331 - val_accuracy: 0.6250 - val_loss: 1.8335 - learning_rate: 0.0010
Epoch 3/100

Epoch 3: val_accuracy did not improve from 0.68750
32/32 - 454s - 14s/step - accuracy: 0.9273 - loss: 0.2008 - val_accuracy: 0.6250 - val_loss: 0.8756 - learning_rate: 0.0010
Epoch 4/100

Epoch 5: val_accuracy did not improve from 0.68750
32/32 - 446s - 14s/step - accuracy: 0.9352 - loss: 0.1887 - val_accuracy: 0.6250 - val_loss: 0.7969 - learning_rate: 0.0010
Epoch 6/100

Epoch 6: val_accuracy did not improve from 0.68750
32/32 - 499s - 16s/step - accuracy: 0.9216 - loss: 0.2029 - val_accuracy: 0.6250 - val_loss: 0.7898 - learning_rate: 0.0010
Epoch 7/100

Epoch 7: val_accuracy did not improve from 0.68750
32/32 - 434s - 14s/step - accuracy: 0.9285 - loss: 0.1789 - val_accuracy: 0.5625 - val_loss: 0.8327 - learning_rate: 0.0010
Epoch 8/100

Epoch 8: val_accuracy did not improve from 0.68750
32/32 - 436s - 14s/step - accuracy: 0.9365 - loss: 0.1683 - val_accuracy: 0.6875 - val_loss: 0.4734 - learning_rate: 0.0010
Epoch 9/100

Epoch 9: val_accuracy did not improve from 0.68750
32/32 - 443s - 14s/step - accuracy: 0.9360 - loss: 0.1759 - val_accuracy: 0.6250 - val_loss: 0.5494 - learning_rate: 0.0010
Epoch 10/100

Epoch 10: val_accuracy did not improve from 0.68750
32/32 - 505s - 16s/step - accuracy: 0.9354 - loss: 0.1648 - val_accuracy: 0.6250 - val_loss: 0.7877 - learning_rate: 0.0010
Epoch 11/100

Epoch 11: val_accuracy did not improve from 0.68750
32/32 - 477s - 15s/step - accuracy: 0.9413 - loss: 0.1511 - val_accuracy: 0.6875 - val_loss: 0.7580 - learning_rate: 0.0010
Epoch 12/100

Epoch 12: val_accuracy improved from 0.68750 to 0.75000, saving model to data/output/models/2026-07-19 05-39-59/12-val_accuracy-0.75-val_loss-0.49.keras

Epoch 12: finished saving model to data/output/models/2026-07-19 05-39-59/12-val_accuracy-0.75-val_loss-0.49.keras
32/32 - 426s - 13s/step - accuracy: 0.9454 - loss: 0.1552 - val_accuracy: 0.7500 - val_loss: 0.4926 - learning_rate: 0.0010
Epoch 13/100

Epoch 13: val_accuracy did not improve from 0.75000

Epoch 13: ReduceLROnPlateau reducing learning rate to 0.00010000000474974513.
32/32 - 407s - 13s/step - accuracy: 0.9473 - loss: 0.1512 - val_accuracy: 0.7500 - val_loss: 0.5936 - learning_rate: 0.0010
Epoch 13: early stopping
Restoring model weights from the end of the best epoch: 8.
Completed Model Training Timestamp: 2026-07-19 07:57:44

C



Te

```
dir_name = r"data/output/models/"
dirs = os.listdir(dir_name)
for i in range(len(dirs)):
    print(i, dirs[i])
```

0 2026-07-19 05-39-59

```
cur_dir = dir_name + dirs[0] + "/"
model_names = os.listdir(cur_dir)
for i in range(len(model_names)):
    print(i, model_names[i])
```

0 01-val_accuracy-0.50-val_loss-0.69.keras
1 12-val_accuracy-0.75-val_loss-0.49.keras
2 04-val_accuracy-0.69-val_loss-0.60.keras

```
print(model_names)
print(len(model_names))
```

```
['01-val_accuracy-0.50-val_loss-0.69.keras', '12-val_accuracy-0.75-val_loss-0.49.keras', '04-val_accuracy-0.69-val_loss-0.60.keras']
3
```

```
model_file = cur_dir + model_names[-1]
```

```
print(model_file)
```

data/output/models/2026-07-19 05-39-59/04-val_accuracy-0.69-val_loss-0.60.keras

 model = keras.models.load_model(model_file)

```

print("Results")

result = model.evaluate(
    test_generator,
    steps=len(test_generator),
    verbose=1
)

print("Loss      : {:.4f}".format(result[0]))
print("Accuracy : {:.2f}%".format(result[1] * 100))

```

```

Results
1/1 ————— 19s 19s/step - accuracy: 0.6522 - loss: 0.6059
Loss      : 0.6059
Accuracy : 65.22%

```

```

print("Results")

y_pred = model.predict(
    test_generator,
    steps=len(test_generator),
    verbose=1
)

y_pred = y_pred.argmax(axis=1)
y_true = test_generator.classes

print("Prediction completed successfully!")

```

```

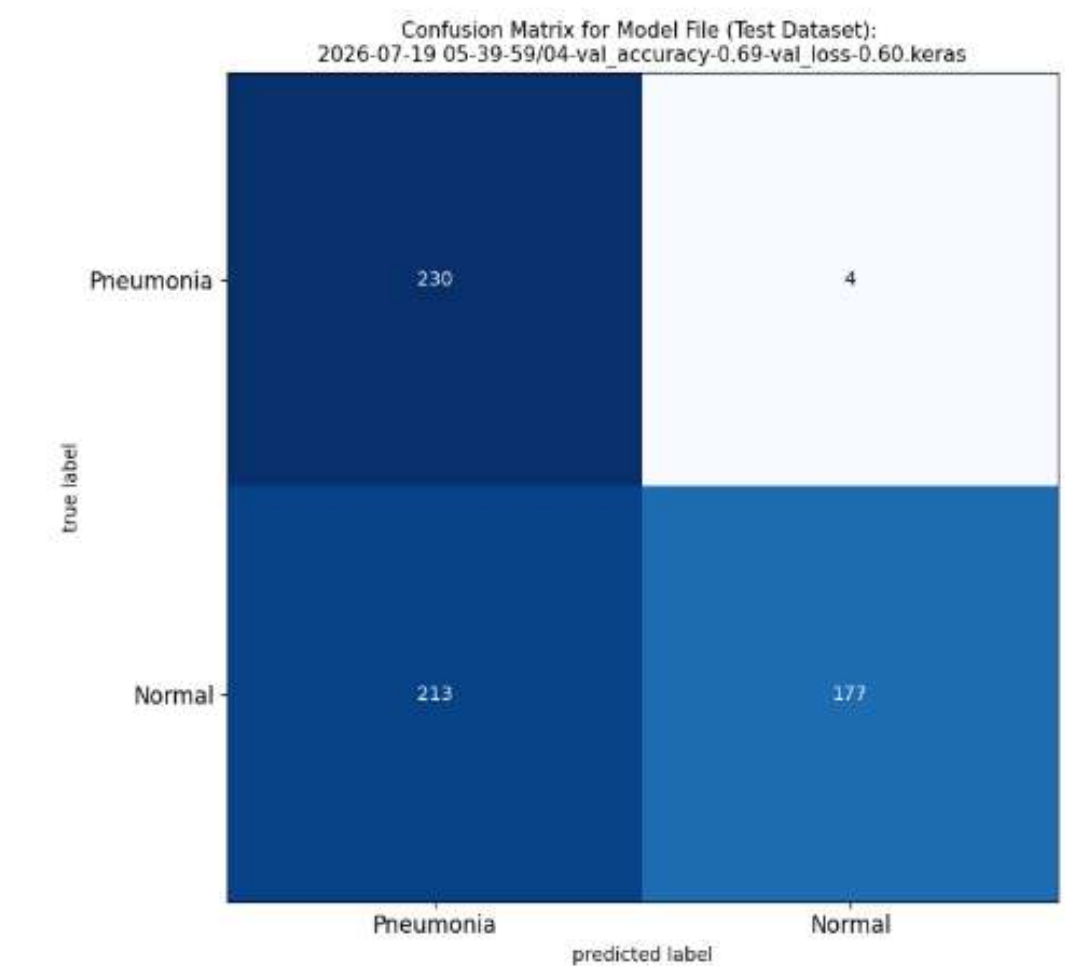
Results
1/1 ————— 17s 17s/step
Prediction completed successfully!

```

```
cls_report = classification_report(y_true, y_pred, target_names=classes, output_dict=True)

print("\n\n")
print("-"*90)
print("Report for Model File: ", model_title)
print("-"*90)
print(cls_report_print)
print("-"*90)
```

Derived Report	
Precision	: 97.79%
Recall	: 45.38%
F1-Score	: 62.00%



Report for Model File: 2026-07-19 05-39-59/04-val_accuracy-0.69-val_loss-0.60.keras

Report for Model File: 2026-07-19 05-39-59/04-val_accuracy-0.69-val_loss-0.60.keras

	precision	recall	f1-score	support
Pneumonia	0.52	0.98	0.68	234
Normal	0.98	0.45	0.62	390
accuracy			0.65	624
macro avg	0.75	0.72	0.65	624
weighted avg	0.81	0.65	0.64	624



```
numofbatch = len(test_generator)

batch_no = random.randint(0, numofbatch-1)

y_img_batch, y_true_batch = test_generator[batch_no]
y_true_batch = y_true_batch.argmax(axis=-1)

y_pred_batch = model.predict(y_img_batch)
y_pred_batch = y_pred_batch.argmax(axis=-1)

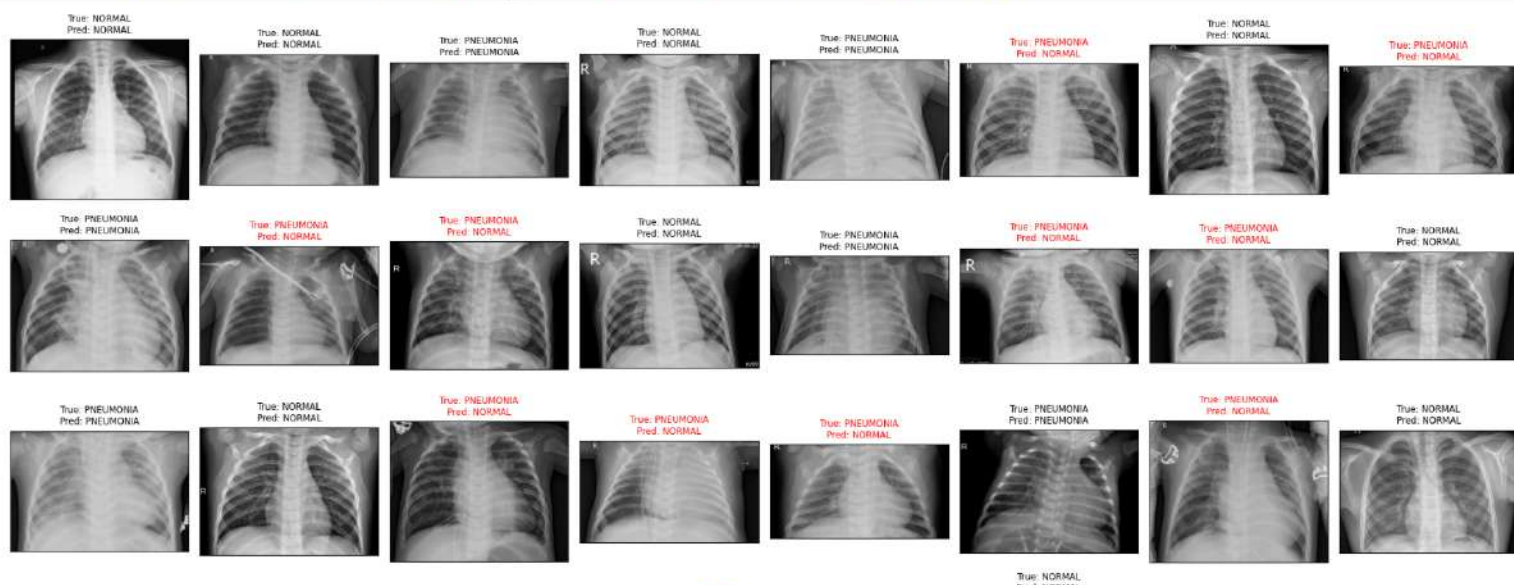
sizeofbatch = len(y_true_batch)
print("-"*35)
print("%s%d" % ("Selected Batch No", batch_no))
print("-"*35)
print("%s%d" % ("Batch Size", len(y_pred_batch)))
print("-"*35)
print("%s%.2f%s" % ("Accuracy", np.mean(y_true==y_pred)*100, "%"))
print("-"*35)
```

... 20/20 ————— 10s 496ms/step

```
-----
Selected Batch No      : 0
-----
Batch Size             : 624
-----
Accuracy               : 65.22%
-----
```



```
show_predictions(y_img_batch, y_true_batch, y_pred_batch, subplot_params, plot_params, class_map, testing_dir, image_file_name_batch, count=count, sample=True)
```



```
ncols = 4
nrows = 2

cols = ncols
rows = nrows

if batch_size_t < 4:
    cols = 1
    rows = batch_size_t

count = cols * rows

subplot_params = get_reset_subplot_params(nrows, ncols, dpi)
plot_params = get_reset_plot_params()

show_predictions(y_ing_batch, y_true_batch, y_pred_batch, subplot_params, plot_params, class_map, testing_dir, image_file_name_sample, count=count, sample=True)
```

...

